Phase 8: BPE 分词器实现指导

1. BPE 训练实现

1.1 整体架构

训练函数的签名：

```
train_bpe(input_path, vocab_size, special_tokens) → (vocab, merges)
```

大致分为三个阶段：

1. **预分词**（I/O 密集，可并行）
2. **初始化 pair 计数**
3. **迭代 merge**（CPU 密集，不可并行，但可增量更新）

1.2 数据结构设计

```
word_freqs: dict[tuple[bytes, ...], int]
  键: pre-token 拆分成的字节元组，如 (b't', b'h', b'e')
  值: 该 pre-token 在语料中的出现次数

pair_counts: dict[tuple[bytes, bytes], int]
  键: 相邻 byte-pair，如 (b't', b'h')
  值: 该 pair 在所有 word 中的总频次（= Σ word 中该 pair 出现次数 × word 频次）

vocab: dict[int, bytes]
  键: token ID
  值: token 对应的字节串

merges: list[tuple[bytes, bytes]]
  按创建顺序记录的 merge 列表
```

1.3 预分词阶段

读取与分块

```
# 使用提供的 find_chunk_boundaries
with open(input_path, "rb") as f:
    boundaries = find_chunk_boundaries(f, num_processes, b"<|endoftext|>")
```

处理单个 chunk 的逻辑

```
对于每个 chunk 文本:
  1. 按 special tokens 分割
     用 re.split(pattern, chunk) 其中 pattern = '|'.join(re.escape(st) for
st in special_tokens)
  2. 对每个非 special 的片段:
     用 re.finditer(PAT, segment) 遍历 pre-tokens
     对每个 pre-token:
       转为 UTF-8 bytes: token.encode("utf-8")
       拆成字节元组: tuple(bytes([b]) for b in encoded)
       在本地 Counter 中计数
  3. 返回本地 Counter
```

并行化

```python
from multiprocessing import Pool
from collections import Counter

def process_chunk(input_path, start, end, special_tokens):
    with open(input_path, "rb") as f:
        f.seek(start)
        chunk = f.read(end - start).decode("utf-8", errors="ignore")
    # 处理 chunk，返回 Counter
    local_counts = Counter()
    # ... 按 special token 分割，finditer，统计 ...
    return local_counts

with Pool(num_processes) as pool:
    results = pool.starmap(process_chunk,
        [(input_path, s, e, special_tokens) for s, e in
zip(boundaries[:-1], boundaries[1:])])

word_freqs = Counter()
for result in results:
    word_freqs += result
```

1.4 初始化 pair 计数

```python
pair_counts = {}
for word, freq in word_freqs.items():
    for i in range(len(word) - 1):
        pair = (word[i], word[i+1])
        pair_counts[pair] = pair_counts.get(pair, 0) + freq
```

1.5 迭代 merge

朴素版本

```
while len(vocab) < target_vocab_size:
    # 找最频繁的 pair（并列时选字典序最大）
    best_pair = max(pair_counts, key=lambda p: (pair_counts[p], p))

    if pair_counts[best_pair] == 0:
        break  # 没有更多可合并的

    # 创建新 token
    new_token = best_pair[0] + best_pair[1]
    vocab[next_id] = new_token
    merges.append(best_pair)

    # 更新 word_freqs 和 pair_counts
    # ... 增量更新 ...
```

增量更新的详细逻辑

当合并 pair (A, B) 时：

```
new_word_freqs = {}
for word, freq in word_freqs.items():
    if pair (A, B) 不在 word 中:
        new_word_freqs[word] = freq
        continue

    # 构造新 word：将所有 (A, B) 合并为 AB
    new_word = 合并 word 中所有相邻的 (A, B)
    new_word_freqs[new_word] = freq

    # 更新 pair_counts:
    # 遍历旧 word，找到所有被影响的 pair
    # 在旧 word 中: ... X A B Y ...
    # 减少: (X,A), (A,B), (B,Y) 各减 freq
    # 在新 word 中: ... X AB Y ...
    # 增加: (X,AB), (AB,Y) 各加 freq

word_freqs = new_word_freqs
```

合并 word 中 pair 的算法

```
给定 word = (t₁, t₂, t₃, ..., tₙ) 和要合并的 pair (A, B):

new_word = []
i = 0
while i < len(word):
    if i < len(word) - 1 and word[i] == A and word[i+1] == B:
        new_word.append(A + B)
```

```
            i += 2
        else:
            new_word.append(word[i])
            i += 1
    return tuple(new_word)
```

1.6 vocab ID 分配

测试对比参考实现，需要确认 ID 分配顺序。从测试代码 `test_train_bpe` 看：

```
# 参考 vocab 的 key 和 value 的集合必须匹配
assert set(vocab.keys()) == set(reference_vocab.keys())
assert set(vocab.values()) == set(reference_vocab.values())
```

这意味着 ID 分配顺序需要与参考一致。查看参考 vocab JSON 可以确认具体顺序。

常见顺序：

1. special tokens（如 `<|endoftext|>`）→ ID 0
2. 256 个单字节 → ID 1~256（或根据 special token 数量偏移）
3. merge 产生的新 token → 按 merge 顺序递增

1.7 性能要求

- `test_train_bpe_speed`：在 `corpus.en` 上训练 500 vocab，< 1.5 秒
- 参考实现 0.38 秒，朴素实现约 3 秒
- 关键优化：增量更新 pair_counts，避免每次全量扫描

---

2. Tokenizer 类

2.1 初始化

```
__init__(self, vocab, merges, special_tokens=None):
  self.vocab = vocab                    # int → bytes
  self.merges = merges                  # list of (bytes, bytes)
  self.special_tokens = special_tokens  # list of str or None

  # 构建反向查找:
  self.vocab_inv = {v: k for k, v in vocab.items()}  # bytes → int

  # 构建 merge 优先级（可选，用于编码加速）:
  self.merge_priority = {merge: i for i, merge in enumerate(merges)}

  # 如果 special_tokens 不在 vocab 中，追加
  if special_tokens:
      for st in special_tokens:
          st_bytes = st.encode("utf-8")
          if st_bytes not in self.vocab_inv:
```

```
                new_id = max(self.vocab.keys()) + 1
                self.vocab[new_id] = st_bytes
                self.vocab_inv[st_bytes] = new_id
```

2.2 encode 方法

```
def encode(self, text: str) -> list[int]:
    # 1. 按 special tokens 分割
    #    需要处理嵌套/重叠的 special tokens
    #    例如 "<|endoftext|><|endoftext|>" 可能是一个 special token
    #    使用 re.split，按最长匹配优先

    # 2. 对每个片段:
    #    如果是 special token → 直接查 vocab_inv
    #    否则 → 预分词 → 对每个 pre-token 应用 merges → 查 vocab_inv

    # 3. 拼接所有 token ID
```

按 special token 分割文本

```
import regex as re

if self.special_tokens:
    # 按长度降序排列，确保更长的 special token 先被匹配
    sorted_tokens = sorted(self.special_tokens, key=len, reverse=True)
    pattern = '(' + '|'.join(re.escape(t) for t in sorted_tokens) + ')'
    parts = re.split(pattern, text)
    # parts 交替为 [非special, special, 非special, special, ...]
else:
    parts = [text]
```

重要：`re.split` 当 pattern 中有捕获组 `()` 时，会保留分隔符。

对 pre-token 应用 merges

```
给定 pre-token 的字节元组 tokens = (b't', b'h', b'e'):

for (A, B) in self.merges:
    new_tokens = []
    i = 0
    while i < len(tokens):
        if i < len(tokens) - 1 and tokens[i] == A and tokens[i+1] == B:
            new_tokens.append(A + B)
            i += 2
        else:
            new_tokens.append(tokens[i])
```

```
                i += 1
        tokens = tuple(new_tokens)

    # 最终 tokens 中的每个元素对应一个 token，查 vocab_inv 得到 ID
```

2.3 decode 方法

```
def decode(self, ids: list[int]) -> str:
    byte_sequence = b''.join(self.vocab[id] for id in ids)
    return byte_sequence.decode("utf-8", errors="replace")
```

2.4 encode_iterable 方法

```
def encode_iterable(self, iterable):
    for text in iterable:
        for token_id in self.encode(text):
            yield token_id
```

关键：使用 `yield` 而非收集到列表——这是测试中内存限制的要求。

2.5 与 tiktoken 对比的重要细节

测试会与 tiktoken 的 GPT-2 encoding 对比。需要注意：

1. **预分词必须完全一致**：使用完全相同的 GPT-2 正则模式
2. **merge 应用顺序必须一致**：按 merges 列表顺序
3. **special token 处理**：先分割文本再预分词，special token 不经过 BPE
4. **空字符串**：`encode("")` 应返回 `[]`

2.6 重叠 special tokens

测试 `test_overlapping_special_tokens` 检查：

```
special_tokens = ["<|endoftext|>", "<|endoftext|><|endoftext|>"]
```

当文本中有 `<|endoftext|><|endoftext|>` 时，应该优先匹配更长的 special token。 这就是为什么按长度降序排列很重要。

---

3. BPE 训练的序列化

3.1 保存 vocab 和 merges

作业要求把 vocab 和 merges 序列化到磁盘。常用格式：

```
import json

# 保存 vocab (JSON)
# 注意: JSON 的 key 必须是 str
# bytes 不能直接 JSON 序列化，需要转换
vocab_serializable = {str(k): list(v) for k, v in vocab.items()}
with open("vocab.json", "w") as f:
    json.dump(vocab_serializable, f)

# 保存 merges (文本文件，每行一个 merge)
with open("merges.txt", "w") as f:
    for a, b in merges:
        # 可以用 GPT-2 风格的编码，或其他方式
        f.write(f"{list(a)} {list(b)}\n")
```

或者直接用 pickle：

```
import pickle
with open("tokenizer.pkl", "wb") as f:
    pickle.dump({"vocab": vocab, "merges": merges}, f)
```

3.2 从文件加载

```
# 对应的加载逻辑
# 如果用了 GPT-2 风格序列化（与测试 fixture 格式兼容）
# 参考 test_tokenizer.py 中 get_tokenizer_from_vocab_merges_path 的逻辑
```

---

4. 常见陷阱

1. **bytes 比较的字典序**：

   ```
   (b'BA', b'A') > (b'B', b'ZZ')  # True，因为 b'BA' > b'B'
   ```

   Python 的 bytes 比较是逐字节、字典序的。

2. **re.split 的行为**：

   ```
   re.split('(x)', 'axbxc')  # ['a', 'x', 'b', 'x', 'c']
   # 有捕获组时，分隔符也会出现在结果中
   ```

3. **空匹配**：`re.split` 可能产生空字符串，需要过滤掉。

4. **bytes 元组 vs bytes 对象**：

```
# 单字节也是 bytes 对象
b'a'  # 这是 bytes，长度为 1
bytes([97])  # 也是 b'a'

# tuple of bytes:
(b'a', b'b', b'c')  # 三个单字节 bytes 组成的元组

# 注意区别:
b'abc'      # 一个 bytes 对象，长度 3
(b'a', b'b', b'c')  # 一个元组，3 个元素
```

5. **merge 后 word 可能缩短到只剩 1 个 token**： 此时该 word 不再贡献任何 pair 计数。

6. **内存**：TinyStories 训练集约 2.1M 文档。

   - word_freqs 可能有数十万个不同的 pre-token
   - 用 Counter 合并比 defaultdict 更方便